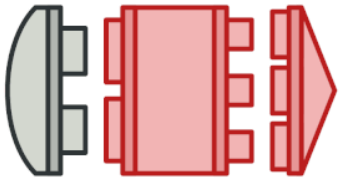
Структурные паттерны. Адаптер



Структурные паттерны

Структурные паттерны проектирования, отвечают за построение удобных в поддержке иерархий классов.

- Использование **Структурных паттернов** помогает из классов и объектов строить более крупные структуры.
- **Структурные паттерны уровня класса** используют наследование для составления композиций из интерфейсов и реализаций.
- **Структурные паттерны уровня объекта** используют композицию объектов для получения новой функциональности. Дополнительная гибкость в этом случае связана с возможностью изменить композицию объектов во время выполнения, что недопустимо для статической композиции классов.



Паттерн Адаптер
Adapter

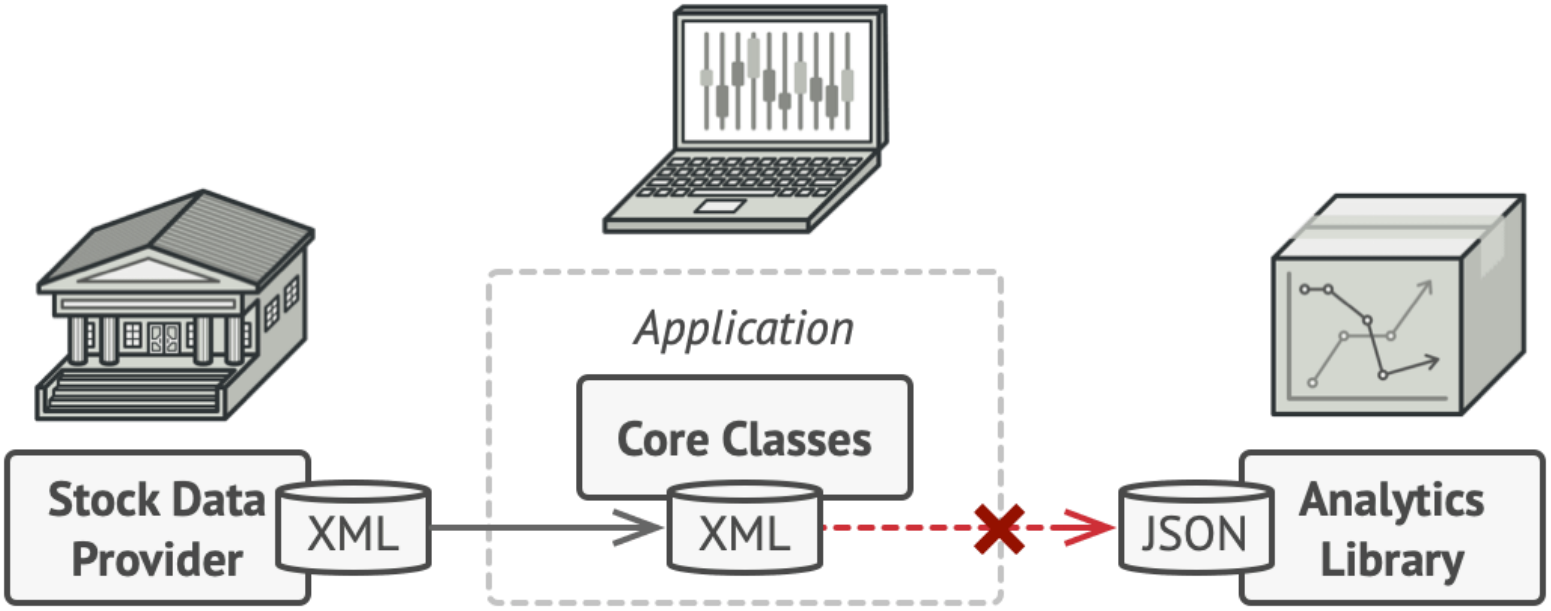


Паттерн **Adapter** - преобразует интерфейс *(набор имен методов*) одного класса в интерфейс (*набор имен методов*) другого класса, который ожидают клиенты.

Адаптер обеспечивает совместную работу классов с несовместимыми интерфейсами.

Проблема

Несовместимость библиотек, поддерживающих разные форматы данных



Переписать библиотеку под другой формат?

- Это может нарушить работу существующего кода, который уже зависит от библиотеки.
- Возможно нет доступа к её исходному коду.

Решение

Создать ***Адаптер*** **-** объект-переводчик, который трансформирует интерфейс или данные одного объекта в такой вид, чтобы он стал понятен другому объекту.

Адаптер «оборачивает» один из объектов, так что другой объект не знает о наличии первого.

Адаптеры могут не только переводить данные из одного формата в другой, но обеспечить взаимодействие объектов с разными интерфейсами.

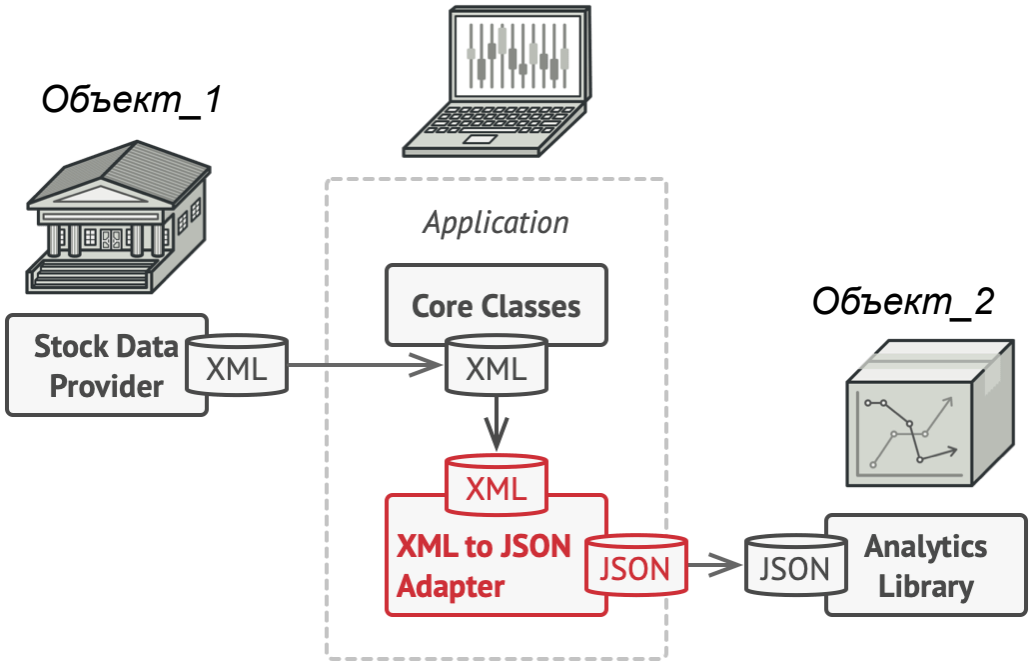
Это работает так:

1.Адаптер имеет интерфейс, который совместим с *Объектом_1*.

2.Этот объект может свободно вызывать методы **Адаптера**.

3.Адаптер получает эти вызовы и перенаправляет их *Объекту_2*, но уже в том формате и последовательности, которые понятны второму объекту.

Возможно создать ***двухсторонний Адаптер***, который работал бы в обе стороны.



Структура паттерна

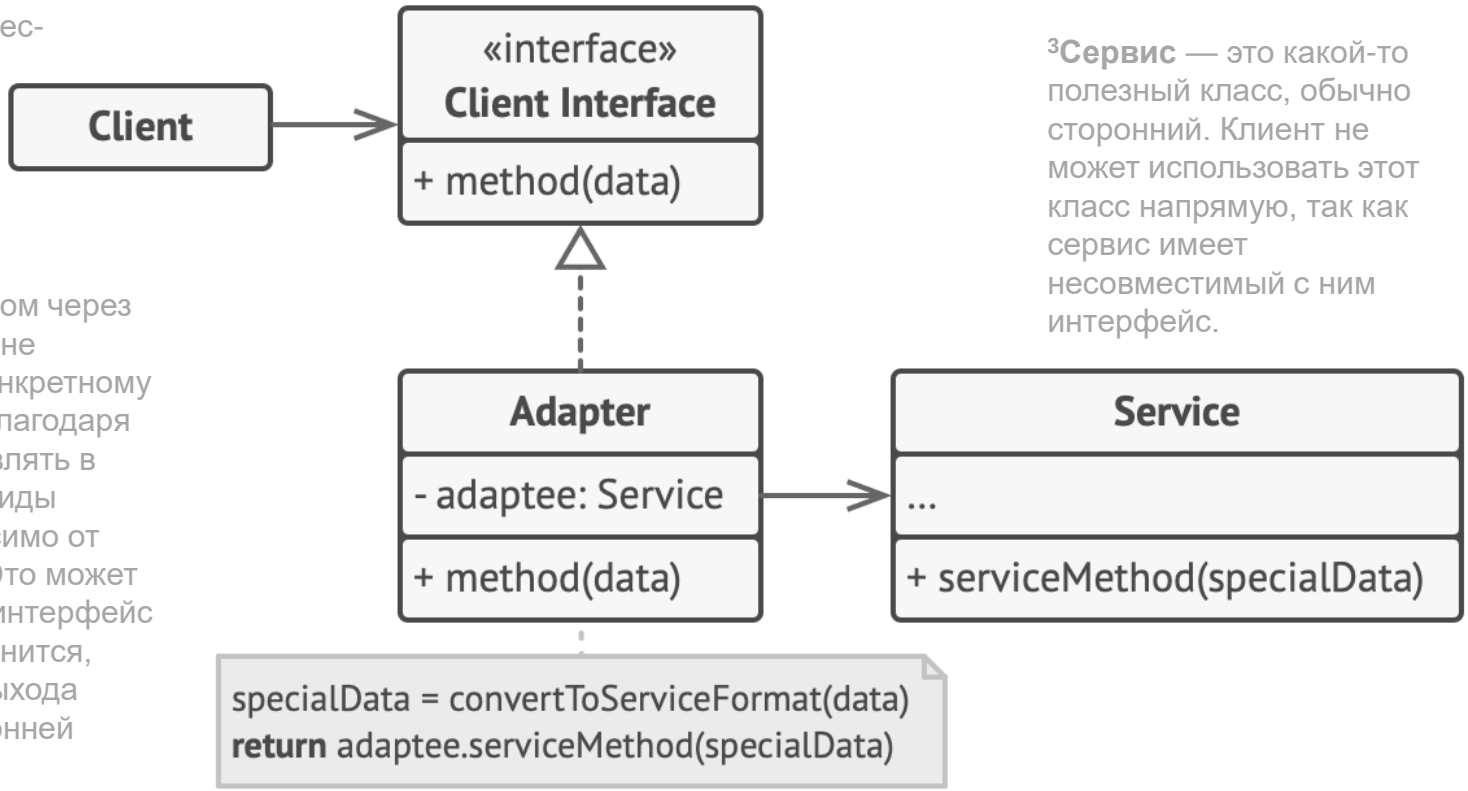
Адаптер объектов

Эта реализация использует агрегацию: объект адаптера «оборачивает», то есть содержит ссылку на служебный объект. Такой подход работает во всех языках программирования.

[1]**Клиент** — это класс, который содержит существующую бизнес-логику программы.

[2]**Клиентский интерфейс** описывает протокол, через который клиент может работать с другими классами.

[3]**Сервис** — это какой-то полезный класс, обычно сторонний. Клиент не может использовать этот класс напрямую, так как сервис имеет несовместимый с ним интерфейс.

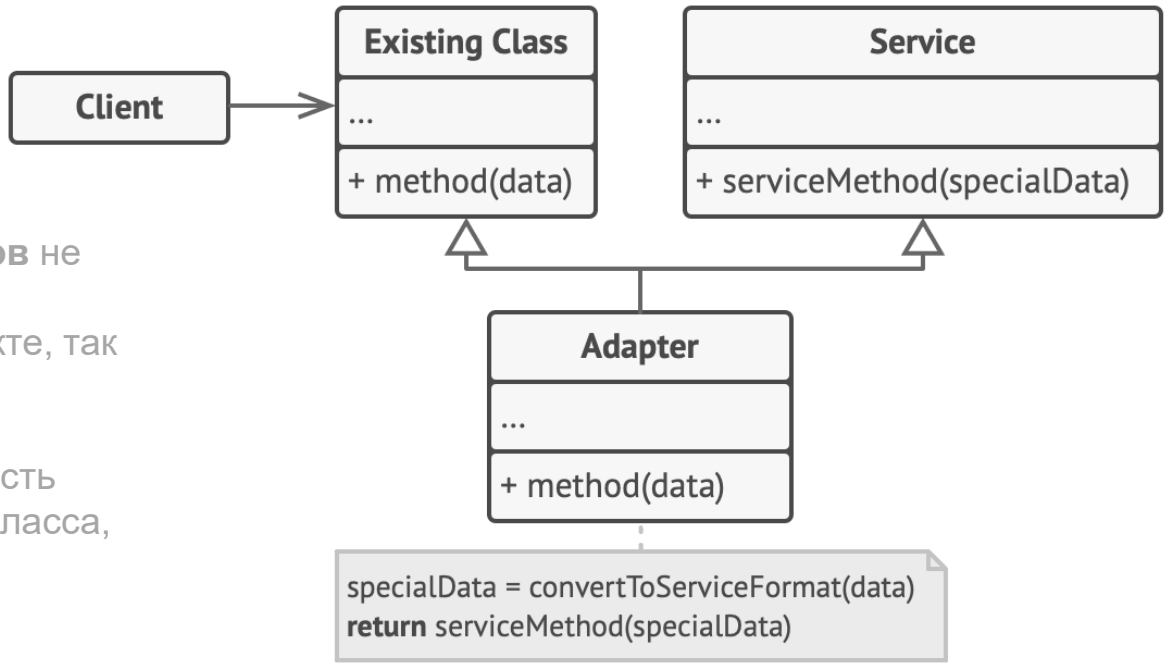


[5]Работая с адаптером через интерфейс, клиент не привязывается к конкретному классу адаптера. Благодаря этому, можно добавлять в программу новые виды адаптеров, независимо от клиентского кода. Это может пригодиться, если интерфейс сервиса вдруг изменится, например, после выхода новой версии сторонней библиотеки.

[4]**Адаптер** — это класс, который может одновременно работать и с клиентом, и с сервисом. Он реализует клиентский интерфейс и содержит ссылку на объект сервиса. Адаптер получает вызовы от клиента через методы клиентского интерфейса, а затем переводит их в вызовы методов обёрнутого объекта в правильном формате.

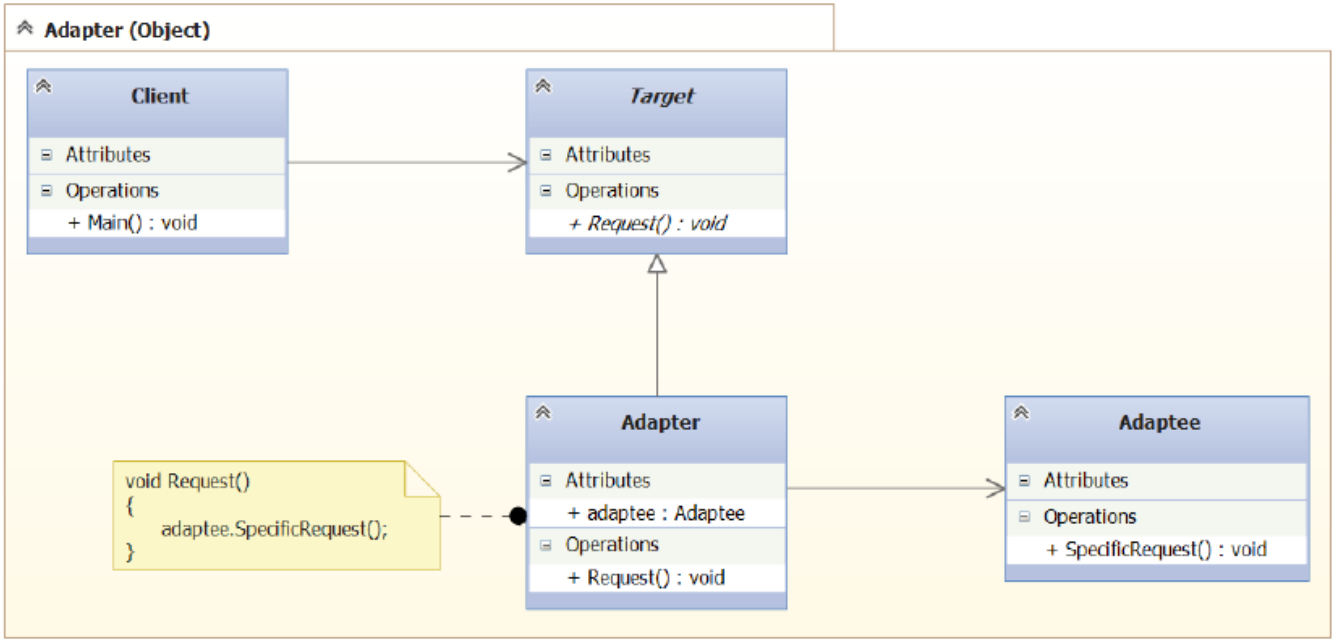
Структура паттерна

Адаптер классов
Эта реализация базируется на наследовании: адаптер наследует оба интерфейса одновременно. Такой подход возможен только в языках, поддерживающих множественное наследование, например, C++.



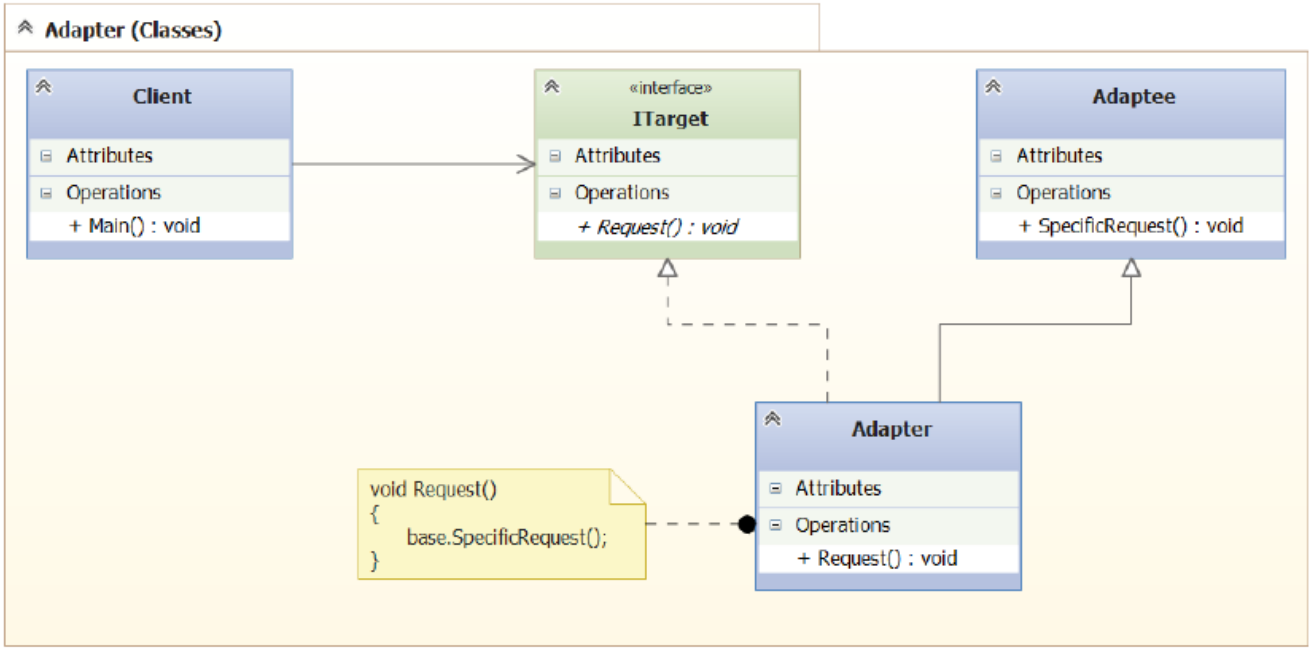
[1]**Адаптер классов** не нуждается во вложенном объекте, так как он может одновременно наследовать и часть существующего класса, и часть сервиса.

Структура паттерна UML



Адаптер уровня объектов

Адаптер уровня классов



Структура паттерна на языке C#

Адаптер уровня объектов

```
class Program
{
    static void Main()
    {
        Target target = new Adapter();
        target.Request();
    }
}
```

```
abstract class Target
{
    public abstract void Request();
}
```

```
class Adapter : Target
{
    Adaptee adaptee = new Adaptee();

    public override void Request()
    {
        adaptee.SpecificRequest();
    }
}
```

```
class Adaptee
{
    public void SpecificRequest()
    {
        Console.WriteLine("SpecificRequest");
    }
}
```

Структура паттерна на языке C#

Адаптер уровня классов

```
class Program
{
    static void Main()
    {
        ITarget target = new Adapter();
        target.Request();
    }
}
```

```
interface ITarget
{
    void Request();
}
```

```
class Adaptee
{
    public void SpecificRequest()
    {
        Console.WriteLine("SpecificRequest");
    }
}
```

```
class Adapter : Adaptee, ITarget
{
    public void Request()
    {
        SpecificRequest();
    }
}
```

Участники

Target - **Цель**:

Формирует требуемый клиенту интерфейс (набор имен методов).

Client - **Клиент**:

Пользуется объектами с интерфейсом Target.

Adaptee - **Адаптируемый**:

Содержит интерфейс (набор методов) требующий адаптации.

Adapter - **Адаптер**

Адаптирует интерфейс Adaptee к интерфейсу Target.

Отношения между участниками

Отношения между классами (для адаптера уровня классов)

Класс Adapter связан связью отношения наследования с классом Adaptee и связью отношения реализации с интерфейсом ITarget.

Отношения между классами (для адаптера уровня объектов)

Класс Adapter связан связью отношения наследования с абстрактным классом Target и связью отношения ассоциации с классом Adaptee.

Отношения между объектами

Клиент вызывает методы на экземпляре класса Adapter. Adapter в свою очередь вызывает методы адаптируемого объекта или класса Adaptee.

Применение

1. Когда есть необходимость использовать сторонний класс, но его интерфейс не соответствует остальному коду приложения.

Адаптер позволяет создать посредника, который будет преобразовывать вызовы приложения в формат, понятный стороннему классу.

2. Когда необходимо использовать несколько существующих подклассов, но в них не хватает какой-то общей функциональности, причём расширить суперкласс невозможно (нет доступа).

Можно создать ещё один уровень подклассов и добавить в них недостающую функциональность. Но при этом придётся дублировать один и тот же код в обеих ветках подклассов.

Шаги реализации

1. Убедиться, что есть два класса с несовместимыми интерфейсами:

- *полезный сервис* — служебный класс, который невозможно изменять (он либо сторонний, либо от него зависит другой код);
- *один или несколько клиентов* — существующие классы приложения, несовместимые с сервисом из-за неудобного или несовпадающего интерфейса.

2. Описать клиентский интерфейс, через который классы приложения смогли бы использовать класс сервиса.

3. Создать класс Адаптера, реализовав этот интерфейс.

4. Поместить в Адаптер поле, которое будет хранить ссылку на объект сервиса. Обычно это поле заполняют объектом, переданным в конструктор Адаптера. В случае простой адаптации этот объект можно передавать через параметры методов адаптера.

5. Реализовать все методы клиентского интерфейса в Адаптере. Адаптер должен делегировать основную работу сервису.

6. Приложение должно использовать Адаптер только через клиентский интерфейс. Это позволит легко изменять и добавлять адаптеры в будущем.

Преимущества и недостатки

+ Отделяет и скрывает от клиента подробности преобразования различных интерфейсов.

- Усложняет код программы из-за введения дополнительных классов.